InternNova Data Analytics Internship

Week 3 Assignment: SQL & Excel for Data Analytics

Submitted by: Brojo Mohan Dutta

Duration: 1 Week (7 Days) | Total Marks: 100

SQL queries for all SQL tasks

Screenshots of SQL query outputs

Excel screenshots for all Excel tasks.

Brief explanation of each task

Dataset (used for Tasks 5–10)

employees.csv

emp_id	name	department_id	age	salary	city	join_year
E004	Sneha Roy	103	31	68000.0	Bangalore	2020
E008	Neha Verma	103	27	65000.0	Bangalore	2022
E005	Karan Mehta	103	40	72000.0	Bangalore	2018
E010	Divya Iyer	104	35	58000.0	Chennai	2019
E009	Arjun Reddy	104	38	60000.0	Chennai	2017
E006	Anjali Gupta	102	28	51000.0	Delhi	2021
E012	Pooja Bansal	999	30	53000.0	Delhi	2021
E001	Aarav Sharma	101	29	42000.0	Kolkata	2021
E011	Rahul Kapoor	101	33	47000.0	Kolkata	2020
E003	Rohan Das	101	26	39000.0	Kolkata	2022
E002	Priya Nair	102	34	55000.0	Mumbai	2019
E007	Vikram Singh	101	45	58000.0	Mumbai	2015

departments.csv

department_id	department_name	head	location
101	Sales	Amit Malhotra	Kolkata
102	Marketing	Ritu Chawla	Delhi
103	IT	Sanjay Kumar	Bangalore
104	Finance	Meera Nair	Chennai
105	HR	Suman Ghosh	Kolkata

Objective

This report develops practical proficiency in SQL and Microsoft Excel for data analytics. It covers writing SELECT queries, filtering with WHERE, sorting with ORDER BY, computing aggregates, grouping data with GROUP BY/HAVING, combining related tables with INNER/LEFT/RIGHT JOINs, and using subqueries to answer comparative questions — followed by Excel-based formatting, conditional logic, lookup functions, and pivot table/chart reporting.

Task 1: Introduction to Databases & SELECT Statement

Explanation: A database organizes related data into tables of rows and columns. This task uses the employees table, retrieving all records and then selecting specific columns with aliases for readability.

SQL Query: `Display all records`

```
-- Display all records
SELECT *
FROM `proud-spring-506309-k9.internnova.employees`;
```

Output Screenshot:

Row	emp_id	name	department_id	age	salary	city	join_year
1	E004	Sneha Roy	103	31	68000.0	Bangalore	2020
2	E008	Neha Verma	103	27	65000.0	Bangalore	2022
3	E005	Karan Mehta	103	40	72000.0	Bangalore	2018
4	E010	Divya Iyer	104	35	58000.0	Chennai	2019
5	E009	Arjun Reddy	104	38	60000.0	Chennai	2017
6	E006	Anjali Gupta	102	28	51000.0	Delhi	2021
7	E012	Pooja Bansal	999	30	53000.0	Delhi	2021
8	E001	Aarav Sharma	101	29	42000.0	Kolkata	2021
9	E011	Rahul Kapoor	101	33	47000.0	Kolkata	2020
10	E003	Rohan Das	101	26	39000.0	Kolkata	2022
11	E002	Priya Nair	102	34	55000.0	Mumbai	2019
12	E007	Vikram Singh	101	45	58000.0	Mumbai	2015

SQL Query: `Select specific columns`

```
-- Select specific columns
SELECT
  name,
  salary
FROM `proud-spring-506309-k9.internnova.employees`;
```

Output Screenshot:

Row	name	salary
1	Sneha Roy	68000.0
2	Neha Verma	65000.0
3	Karan Mehta	72000.0
4	Divya Iyer	58000.0
5	Arjun Reddy	60000.0
6	Anjali Gupta	51000.0
7	Pooja Bansal	53000.0
8	Aarav Sharma	42000.0
9	Rahul Kapoor	47000.0
10	Rohan Das	39000.0
11	Priya Nair	55000.0
12	Vikram Singh	58000.0

SQL Query: `Select specific columns with aliases`

```
-- Select specific columns with aliases
SELECT
  name AS employee_name,
  salary AS monthly_salary
FROM `proud-spring-506309-k9.internnova.employees`;
```

Output Screenshot:

Row	employee_name	monthly_salary
1	Sneha Roy	68000.0
2	Neha Verma	65000.0
3	Karan Mehta	72000.0
4	Divya Iyer	58000.0
5	Arjun Reddy	60000.0
6	Anjali Gupta	51000.0
7	Pooja Bansal	53000.0
8	Aarav Sharma	42000.0
9	Rahul Kapoor	47000.0
10	Rohan Das	39000.0
11	Priya Nair	55000.0
12	Vikram Singh	58000.0

Task 2: WHERE, ORDER BY & Aggregate Functions

Explanation: Filters employees earning above ₹55,000 using WHERE with a comparison operator, sorts all employees by salary descending, and computes COUNT, SUM, AVG, MIN, and MAX across the whole table.

SQL Query: `WHERE with comparison operator`

```
-- WHERE with comparison operator
SELECT name, Department ID, salary
FROM `proud-spring-506309-k9.internnova.employees`
WHERE salary > 55000;
```

Output Screenshot:

Row	name	department_id	salary
1	Sneha Roy	103	68000.0
2	Neha Verma	103	65000.0
3	Karan Mehta	103	72000.0
4	Divya Iyer	104	58000.0
5	Arjun Reddy	104	60000.0
6	Vikram Singh	101	58000.0

SQL Query: `ORDER BY (descending)`

```
-- ORDER BY (descending)
SELECT name, salary
FROM `proud-spring-506309-k9.internnova.employees`
ORDER BY salary DESC;
```

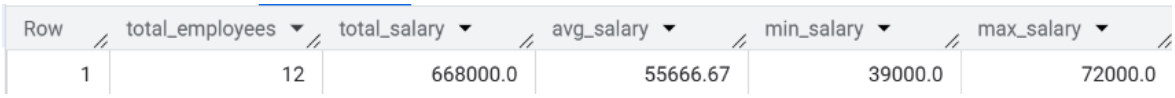
Output Screenshot:

Row	name	salary
1	Karan Mehta	72000.0
2	Sneha Roy	68000.0
3	Neha Verma	65000.0
4	Arjun Reddy	60000.0
5	Divya Iyer	58000.0
6	Vikram Singh	58000.0
7	Priya Nair	55000.0
8	Pooja Bansal	53000.0
9	Anjali Gupta	51000.0
10	Rahul Kapoor	47000.0
11	Aarav Sharma	42000.0
12	Rohan Das	39000.0

SQL Query: `Aggregate functions`

```
-- Aggregate functions
SELECT
  COUNT(*) AS total_employees,
  SUM(salary) AS total_salary,
  ROUND(AVG(salary),2) AS avg_salary,
  MIN(salary) AS min_salary,
  MAX(salary) AS max_salary
FROM `proud-spring-506309-k9.internnova.employees`;
```

Output Screenshot:



Row	total_employees	total_salary	avg_salary	min_salary	max_salary
1	12	668000.0	55666.67	39000.0	72000.0

Task 3: GROUP BY & HAVING

Explanation: Groups employees by Department ID to get per-department headcount and average salary, then uses HAVING to keep only departments with more than 2 employees (HAVING filters after grouping, unlike WHERE which filters rows before grouping).

SQL Query: `GROUP BY`

```
-- GROUP BY
SELECT
  Department ID,
  COUNT(*) AS emp_count,
  ROUND(AVG(salary), 2) AS avg_salary
FROM `proud-spring-506309-k9.internnova.employees`
GROUP BY Department ID;
```

Output Screenshot:

Row	department_id	emp_count	avg_salary
1	103	3	68333.33
2	104	2	59000.0
3	102	2	53000.0
4	999	1	53000.0
5	101	4	46500.0

SQL Query: `GROUP BY + HAVING`

```
-- GROUP BY + HAVING
SELECT
  Department ID,
  COUNT(*) AS emp_count,
  ROUND(AVG(salary), 2) AS avg_salary
FROM `proud-spring-506309-k9.internnova.employees`
GROUP BY Department ID
HAVING COUNT(*) > 2;
```

Output Screenshot:

Row	department_id	emp_count	avg_salary
1	103	3	68333.33
2	101	4	46500.0

Task 4: SQL Joins

Explanation: Joins `employees` and `departments` on `Department ID`.

INNER JOIN — returns only rows with a match in both tables (excludes emp 12's unmatched dept 999 and dept 105 "HR" with no employees).

SQL Query: `INNER JOIN`

```
-- INNER JOIN
SELECT e.name, e.salary, d.department_name, d.location
FROM `proud-spring-506309-k9.internnova.employees` e
INNER JOIN `proud-spring-506309-k9.internnova.departments` d
  ON e.Department ID = d.Department ID;
```

Output Screenshot:

Row	name	salary	department_name	location
1	Sneha Roy	68000.0	IT	Bangalore
2	Neha Verma	65000.0	IT	Bangalore
3	Karan Mehta	72000.0	IT	Bangalore
4	Divya Iyer	58000.0	Finance	Chennai
5	Arjun Reddy	60000.0	Finance	Chennai
6	Anjali Gupta	51000.0	Marketing	Delhi
7	Aarav Sharma	42000.0	Sales	Kolkata
8	Rahul Kapoor	47000.0	Sales	Kolkata
9	Rohan Das	39000.0	Sales	Kolkata
10	Priya Nair	55000.0	Marketing	Delhi
11	Vikram Singh	58000.0	Sales	Kolkata

LEFT JOIN — keeps all employees, with NULLs where no matching department exists (shows emp 12 with a NULL department name).

SQL Query: `LEFT JOIN`

```
-- LEFT JOIN
SELECT e.name, e.Department ID, d.department_name
```

```
FROM `proud-spring-506309-k9.internnova.employees` e
LEFT JOIN `proud-spring-506309-k9.internnova.departments` d
  ON e.Department ID = d.Department ID;
```

Output Screenshot:

Row	name	department_id	department_name
1	Sneha Roy	103	IT
2	Neha Verma	103	IT
3	Karan Mehta	103	IT
4	Divya Iyer	104	Finance
5	Arjun Reddy	104	Finance
6	Anjali Gupta	102	Marketing
7	Pooja Bansal	999	*null*
8	Aarav Sharma	101	Sales
9	Rahul Kapoor	101	Sales
10	Rohan Das	101	Sales
11	Priya Nair	102	Marketing
12	Vikram Singh	101	Sales

RIGHT JOIN — keeps all departments, with NULLs where no matching employee exists (shows "HR" with a NULL employee name).

SQL Query: `RIGHT JOIN`

```
-- RIGHT JOIN
SELECT e.name, d.department_name, d.location
FROM `proud-spring-506309-k9.internnova.employees` e
RIGHT JOIN `proud-spring-506309-k9.internnova.departments` d
  ON e.Department ID = d.Department ID;
```

Output Screenshot:

Row	name	department_name	location
1	Sneha Roy	IT	Bangalore
2	Neha Verma	IT	Bangalore
3	Karan Mehta	IT	Bangalore
4	Divya Iyer	Finance	Chennai
5	Arjun Reddy	Finance	Chennai
6	Anjali Gupta	Marketing	Delhi
7	Aarav Sharma	Sales	Kolkata
8	Rahul Kapoor	Sales	Kolkata
9	Rohan Das	Sales	Kolkata
10	Priya Nair	Marketing	Delhi
11	Vikram Singh	Sales	Kolkata
12	*null*	HR	Kolkata

Task 5: SQL Subqueries

Explanation: First query finds employees earning more than the company-wide average salary using a scalar subquery. Second query finds the highest-paid employee within each department using a subquery with GROUP BY inside an IN clause.

SQL Query: `Employees earning more than the average salary`

```
-- Employees earning more than the average salary
SELECT name, salary
FROM `proud-spring-506309-k9.internnova.employees`
WHERE salary > (
  SELECT AVG(salary) FROM `proud-spring-506309-k9.internnova.employees`
);
```

Output Screenshot:

Row	name	salary
1	Sneha Roy	68000.0
2	Neha Verma	65000.0
3	Karan Mehta	72000.0
4	Divya Iyer	58000.0
5	Arjun Reddy	60000.0
6	Vikram Singh	58000.0

SQL Query: `Highest-paid employee per department`

```
-- Highest-paid employee per department
SELECT name, Department ID, salary
```

```
FROM `proud-spring-506309-k9.internnova.employees`
WHERE salary IN (
  SELECT MAX(salary)
  FROM `proud-spring-506309-k9.internnova.employees`
  GROUP BY Department ID
);
```

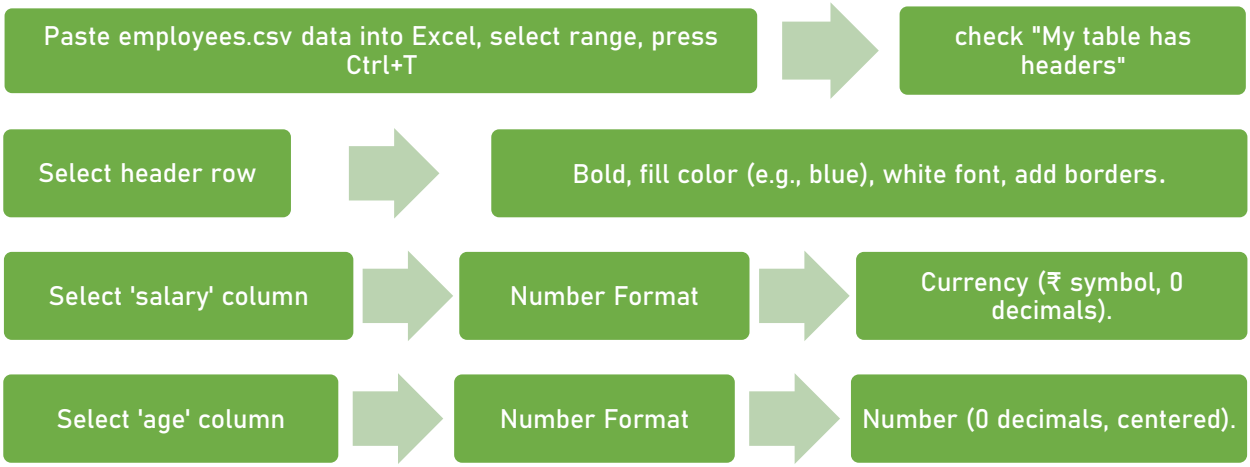
Output Screenshot:

Row	name	department_id	salary
1	Karan Mehta	103	72000.0
2	Divya Iyer	104	58000.0
3	Arjun Reddy	104	60000.0
4	Pooja Bansal	999	53000.0
5	Priya Nair	102	55000.0
6	Vikram Singh	101	58000.0

Task 6: Data Formatting, Sorting & Filtering

Explanation: Load the employees dataset into Excel as a proper Table (Ctrl+T), apply header formatting (bold, fill color, borders), format the salary column as currency, sort by salary, then apply filters to isolate specific records.

Steps:

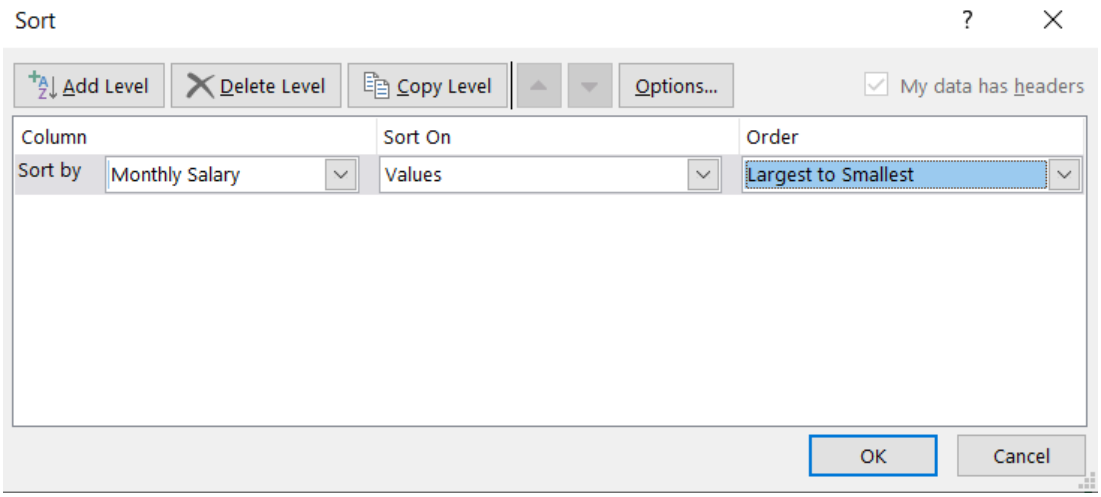


Output Screenshot:

	A	B	C	D	E	F	G
1	Employee ID	Employee Name	Department ID	Age	Monthly Salary	City	Join Year
2	E004	Sneha Roy	103	31	₹ 68,000	Bangalore	2020
3	E008	Neha Verma	103	27	₹ 65,000	Bangalore	2022
4	E005	Karan Mehta	103	40	₹ 72,000	Bangalore	2018
5	E010	Divya Iyer	104	35	₹ 58,000	Chennai	2019
6	E009	Arjun Reddy	104	38	₹ 60,000	Chennai	2017
7	E006	Anjali Gupta	102	28	₹ 51,000	Delhi	2021
8	E012	Pooja Bansal	999	30	₹ 53,000	Delhi	2021
9	E001	Aarav Sharma	101	29	₹ 42,000	Kolkata	2021
10	E011	Rahul Kapoor	101	33	₹ 47,000	Kolkata	2020
11	E003	Rohan Das	101	26	₹ 39,000	Kolkata	2022
12	E002	Priya Nair	102	34	₹ 55,000	Mumbai	2019
13	E007	Vikram Singh	101	45	₹ 58,000	Mumbai	2015

Steps:

Output Screenshot:



	A	B	C	D	E	F	G
1	Employee ID	Employee Name	Department ID	Age	Monthly Salary	City	Join Year
2	E005	Karan Mehta	103	40	₹ 72,000	Bangalore	2018
3	E004	Sneha Roy	103	31	₹ 68,000	Bangalore	2020
4	E008	Neha Verma	103	27	₹ 65,000	Bangalore	2022
5	E009	Arjun Reddy	104	38	₹ 60,000	Chennai	2017
6	E010	Divya Iyer	104	35	₹ 58,000	Chennai	2019
7	E007	Vikram Singh	101	45	₹ 58,000	Mumbai	2015
8	E002	Priya Nair	102	34	₹ 55,000	Mumbai	2019
9	E012	Pooja Bansal	999	30	₹ 53,000	Delhi	2021
10	E006	Anjali Gupta	102	28	₹ 51,000	Delhi	2021
11	E011	Rahul Kapoor	101	33	₹ 47,000	Kolkata	2020
12	E001	Aarav Sharma	101	29	₹ 42,000	Kolkata	2021
13	E003	Rohan Das	101	26	₹ 39,000	Kolkata	2022

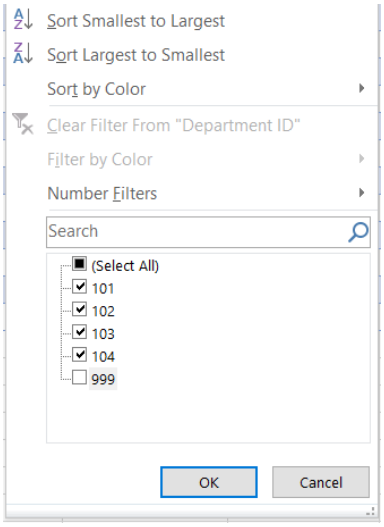
Steps:

Data tab → Filter (or Ctrl+Shift+L) to add dropdown arrows to each header.

Apply filter: Department ID column → uncheck all except 103 (IT) to isolate IT department records

Apply filter: salary column → Number Filters → Greater Than 55000

Output Screenshot:

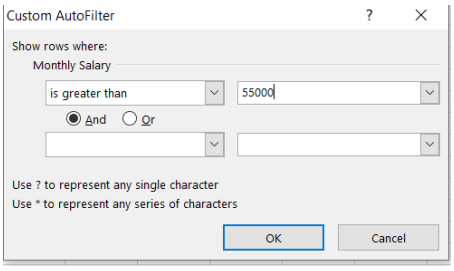


	A	B	C	D	E	F	G
1	Employee ID	Employee Name	Department ID	Age	Monthly Salary	City	Join Year
2	E005	Karan Mehta	103	40	₹ 72,000	Bangalore	2018
3	E004	Sneha Roy	103	31	₹ 68,000	Bangalore	2020
4	E008	Neha Verma	103	27	₹ 65,000	Bangalore	2022
5	E009	Arjun Reddy	104	38	₹ 60,000	Chennai	2017
6	E010	Divya Iyer	104	35	₹ 58,000	Chennai	2019
7	E007	Vikram Singh	101	45	₹ 58,000	Mumbai	2015
8	E002	Priya Nair	102	34	₹ 55,000	Mumbai	2019
10	E006	Anjali Gupta	102	28	₹ 51,000	Delhi	2021
11	E011	Rahul Kapoor	101	33	₹ 47,000	Kolkata	2020
12	E001	Aarav Sharma	101	29	₹ 42,000	Kolkata	2021
13	E003	Rohan Das	101	26	₹ 39,000	Kolkata	2022

Steps:

Apply filter: salary column → Number Filters → Greater Than 55000

Output Screenshot:



	A	B	C	D	E	F	G
1	Employee ID	Employee Name	Department ID	Age	Monthly Salary	City	Join Year
2	E005	Karan Mehta	103	40	₹ 72,000	Bangalore	2018
3	E004	Sneha Roy	103	31	₹ 68,000	Bangalore	2020
4	E008	Neha Verma	103	27	₹ 65,000	Bangalore	2022
5	E009	Arjun Reddy	104	38	₹ 60,000	Chennai	2017
6	E010	Divya Iyer	104	35	₹ 58,000	Chennai	2019
7	E007	Vikram Singh	101	45	₹ 58,000	Mumbai	2015

Task 7: Conditional Formatting & Excel Functions

Explanation: Conditional formatting visually highlights values meeting a rule. IF() returns one value or another based on a test, COUNTIF() counts cells matching a condition, and SUMIF() sums values matching a condition — all applied to the employees dataset.

Steps:

Output Screenshot:

	A	B	C	D	E	F	G
1	Employee ID	Employee Name	Department ID	Age	Monthly Salary	City	Join Year
2	E005	Karan Mehta	103	40	₹ 72,000	Bangalore	2018
3	E004	Sneha Roy	103	31	₹ 68,000	Bangalore	2020
4	E008	Neha Verma	103	27	₹ 65,000	Bangalore	2022
5	E009	Arjun Reddy	104	38	₹ 60,000	Chennai	2017
6	E010	Divya Iyer	104	35	₹ 58,000	Chennai	2019
7	E007	Vikram Singh	101	45	₹ 58,000	Mumbai	2015
8	E002	Priya Nair	102	34	₹ 55,000	Mumbai	2019
9	E012	Pooja Bansal	999	30	₹ 53,000	Delhi	2021
10	E006	Anjali Gupta	102	28	₹ 51,000	Delhi	2021
11	E011	Rahul Kapoor	101	33	₹ 47,000	Kolkata	2020
12	E001	Aarav Sharma	101	29	₹ 42,000	Kolkata	2021
13	E003	Rohan Das	101	26	₹ 39,000	Kolkata	2022

Formulas:

```
IF() - flag high earners(Insert new column F - "Category”):
=IF(E2>60000,"High Earner",IF(E2<45000,"Low Earner","Standard"))
-- E2 refers to the salary cell in row 2; copy down the column.
```

Output Screenshot:

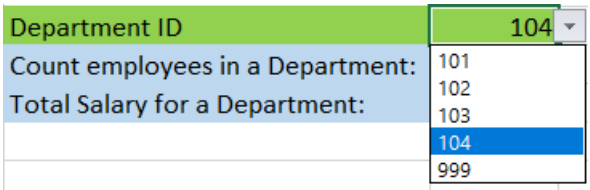
	A	B	C	D	E	F	G	H
1	Employee ID	Employee Name	Department ID	Age	Monthly Salary	Category	City	Join Year
2	E005	Karan Mehta	103	40	₹ 72,000	High Earner	Bangalore	2018
3	E004	Sneha Roy	103	31	₹ 68,000	High Earner	Bangalore	2020
4	E008	Neha Verma	103	27	₹ 65,000	High Earner	Bangalore	2022
5	E009	Arjun Reddy	104	38	₹ 60,000	Standard	Chennai	2017
6	E010	Divya Iyer	104	35	₹ 58,000	Standard	Chennai	2019
7	E007	Vikram Singh	101	45	₹ 58,000	Standard	Mumbai	2015
8	E002	Priya Nair	102	34	₹ 55,000	Standard	Mumbai	2019
9	E012	Pooja Bansal	999	30	₹ 53,000	Standard	Delhi	2021
10	E006	Anjali Gupta	102	28	₹ 51,000	Standard	Delhi	2021
11	E011	Rahul Kapoor	101	33	₹ 47,000	Standard	Kolkata	2020
12	E001	Aarav Sharma	101	29	₹ 42,000	Low Earner	Kolkata	2021
13	E003	Rohan Das	101	26	₹ 39,000	Low Earner	Kolkata	2022

Formulas:

```
COUNTIF() - count employees in a department:
=COUNTIF(C2:C13,N18)
-- Counts how many employees have Department ID = 101 (Sales).

SUMIF() - total salary for a department:
=SUMIF(C2:C13,N18,G2:G13)
-- Sums the salary column (E) where Department ID (C) equals 103 (IT).
```

Output Screenshot:



Department ID	104
Count employees in a Department:	2
Total Salary for a Department:	118000

Task 8: VLOOKUP & XLOOKUP

Explanation: Uses the two related tables (employees and departments) to pull department_name into the employees sheet by matching Department ID. VLOOKUP requires the lookup column to be the leftmost column in the range and only searches left-to-right; XLOOKUP has no such restriction and can also return a custom value when no match is found (useful for Department ID = 999)..

Steps:

```
Assume 'departments' table is on Sheet2, columns A:D (Department ID, department_name, head,
location).
Assume 'employees' table is on Sheet1, Department ID in column C.

VLOOKUP (in Sheet1, new column F - "Dept Name (VLOOKUP)"):
=VLOOKUP([@[Department ID]],departments!$A$2:$B$6,2,FALSE)
-- Looks up C2 (Department ID) in Sheet2 column A, returns the 2nd column
(department_name), exact match.
-- Result for emp_id 12 (Department ID 999): returns #N/A since no match exists.
```

Output Screenshot

	A	B	C	D	E	F	G	H	I
1	Employee ID	Employee Name	Department ID	Dept Name (VLOOKUP)	Age	Monthly Salary	Category	City	Join Year
2	E005	Karan Mehta	103	IT	40	₹ 72,000	High Earner	Bangalore	2018
3	E004	Sneha Roy	103	IT	31	₹ 68,000	High Earner	Bangalore	2020
4	E008	Neha Verma	103	IT	27	₹ 65,000	High Earner	Bangalore	2022
5	E009	Arjun Reddy	104	Finance	38	₹ 60,000	Standard	Chennai	2017
6	E010	Divya Iyer	104	Finance	35	₹ 58,000	Standard	Chennai	2019
7	E007	Vikram Singh	101	Sales	45	₹ 58,000	Standard	Mumbai	2015
8	E002	Priya Nair	102	Marketing	34	₹ 55,000	Standard	Mumbai	2019
9	E012	Pooja Bansal	999	#N/A	30	₹ 53,000	Standard	Delhi	2021
10	E006	Anjali Gupta	102	Marketing	28	₹ 51,000	Standard	Delhi	2021
11	E011	Rahul Kapoor	101	Sales	33	₹ 47,000	Standard	Kolkata	2020
12	E001	Aarav Sharma	101	Sales	29	₹ 42,000	Low Earner	Kolkata	2021
13	E003	Rohan Das	101	Sales	26	₹ 39,000	Low Earner	Kolkata	2022

Steps:

```
XLOOKUP (in Sheet1, new column G - "Dept Name (XLOOKUP)"):
=XLOOKUP(C2,departments!$A$2:$A$6,departments!$B$2:$B$6,"Not Found")
-- Looks up C2 in Sheet2 column A, returns matching value from column B.
-- Result for emp_id 12 (Department ID 999): returns "Not Found" instead of an error, since
a default value is specified.
```

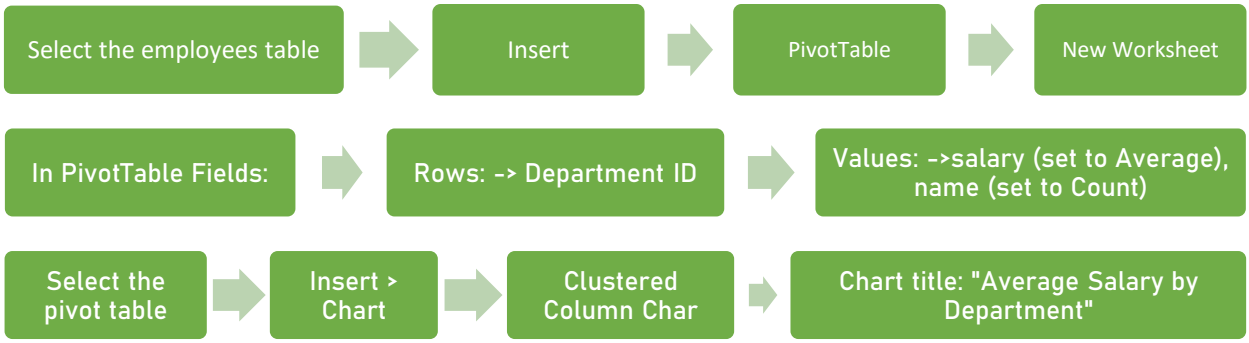
Output Screenshot:

	A	B	C	D	E	F	G	H	I	J
1	Employee ID	Employee Name	Department ID	Dept Name (VLOOKUP)	Dept Name (XLOOKUP)	Age	Monthly Salary	Category	City	Join Year
2	E005	Karan Mehta	103	IT	IT	40	₹ 72,000	High Earner	Bangalore	2018
3	E004	Sneha Roy	103	IT	IT	31	₹ 68,000	High Earner	Bangalore	2020
4	E008	Neha Verma	103	IT	IT	27	₹ 65,000	High Earner	Bangalore	2022
5	E009	Arjun Reddy	104	Finance	Finance	38	₹ 60,000	Standard	Chennai	2017
6	E010	Divya Iyer	104	Finance	Finance	35	₹ 58,000	Standard	Chennai	2019
7	E007	Vikram Singh	101	Sales	Sales	45	₹ 58,000	Standard	Mumbai	2015
8	E002	Priya Nair	102	Marketing	Marketing	34	₹ 55,000	Standard	Mumbai	2019
9	E012	Pooja Bansal	999	#N/A	Not Found	30	₹ 53,000	Standard	Delhi	2021
10	E006	Anjali Gupta	102	Marketing	Marketing	28	₹ 51,000	Standard	Delhi	2021
11	E011	Rahul Kapoor	101	Sales	Sales	33	₹ 47,000	Standard	Kolkata	2020
12	E001	Aarav Sharma	101	Sales	Sales	29	₹ 42,000	Low Earner	Kolkata	2021
13	E003	Rohan Das	101	Sales	Sales	26	₹ 39,000	Low Earner	Kolkata	2022

Task 9: Pivot Table & Charts

Explanation: Summarizes the employees dataset by department using a Pivot Table, then visualizes it with a chart.

Steps:



Output Screenshot:

	A	B
1	**Average Salary by Department**	
2		
3	**Row Labels**	**Average of Monthly Salary**
4	Finance	₹ 59,000
5	IT	₹ 68,333
6	Marketing	₹ 53,000
7	Not Found	₹ 53,000
8	Sales	₹ 46,500
9	**Grand Total**	**₹ 55,667**

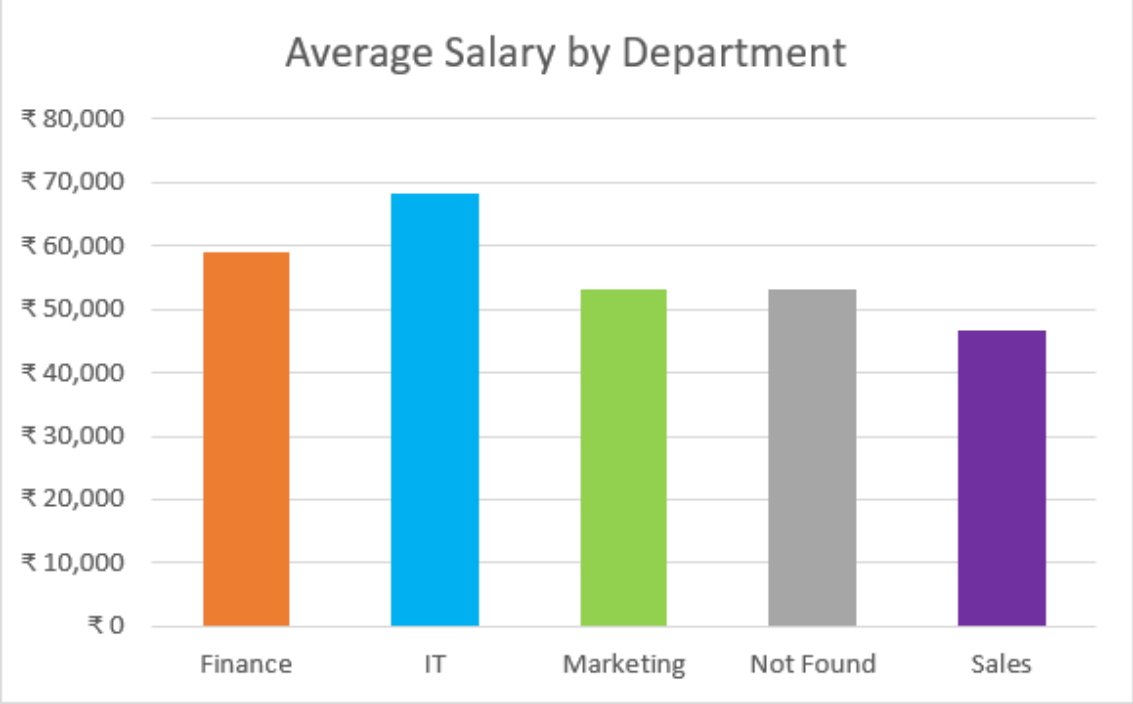


Chart explanation: The clustered column chart shows average salary per department side by side, making it easy to see at a glance that **IT** has the highest average pay and **Sales** has the largest headcount — supporting quick department-level compensation comparisons.